aspects of Elasticsearch, it will benefit us in the long run to understand the high-level architecture, the inner workings of the server, and the dichotomy of its moving parts.

Getting a grip on the server is highly recommended for engineers wishing to use Elasticsearch effectively and efficiently. We may need to debug why query results are not what we expect. We may be tasked with finding the reasons for performance degradation arising from the exponential growth of indexed data. The cluster may become unstable due to memory problems, leading to a 2:00 a.m. production call. Or the business requirements may call for a custom language analyzer to integrate the application with another country's language.

As engineers, we are expected to be able to fine-tune queries or tweak administrative features or spin up an Elasticsearch multicluster farm for a business need. And to gain such knowledge, we must master the technology by understanding its internal workings—its bolts and bearings—which is what this chapter is all about.

In this chapter, we talk through the building blocks of Elasticsearch and gain a better understanding of how the searching and indexing process works behind the scenes. We learn about fundamentals that underlie the search engine, such as inverted indexes, relevancy, and text analysis. Finally, we explore the clustering and distributed nature of the Elasticsearch server. First, let's look at how the Elasticsearch engine works from a high level.

3.1 A high-level overview

Elasticsearch is a server-side application capable of running on anything from personal PCs to a farm of computers serving gigabytes, terabytes, or petabytes of data. It was developed using the Java programming language with Apache Lucene under the hood.

Apache Lucene, a high-performance, full-text searching library developed in Java, is well-known for its powerful searching and indexing features. However, Lucene is not a complete application that we can simply download, install, and work with. As it is a library, we are expected to integrate applications with it via programming interfaces. Elasticsearch does exactly that: it wraps Lucene as its core full-text search library, building a distributed and scalable server-side application. Elasticsearch built a programming language-agnostic application with Lucene as the centerpiece for serving full-text searches.

Elasticsearch is more than just a full-text search engine, however. It has grown into a popular search engine with aggregations and analytics that serve various use cases (application monitoring, log data analytics, web app search functions, security event capturing, machine learning, and more). It is a performant, scalable, modern search engine with high availability, fault tolerance, and speed as its primary goals.

Just as a bank account is useless without money in it, Elasticsearch is nothing without data: the data that goes into Elasticsearch and the data that comes out. Let's spend some time looking at how Elasticsearch treats this data.